

Guia completo — Documentação viva no BookShelf-API

Objetivo

Neste tutorial, você vai configurar um fluxo de documentação viva no projeto **BookShelf-API**.

A ideia é que, sempre que uma mudança for feita no código, o projeto consiga gerar automaticamente um prompt com o `git diff`. Esse prompt será usado no Kiro ou GitHub Copilot para atualizar apenas a documentação realmente impactada.

Neste desafio, a mudança principal será uma nova regra de negócio:

Livros com status reading não podem ser removidos diretamente.

A documentação viva deverá avaliar se essa mudança impacta:

README.md
docs/INSTALLATION.md
docs/diagrams/api-flow.md

Nem todos os documentos precisam ser alterados. A decisão deve ser baseada no impacto real da mudança.

1. Clonar o repositório a partir da branch develop

Nesta etapa, você vai baixar o projeto **BookShelf-API** já usando a branch `develop` como base da atividade.

A branch `develop` será o ponto de partida do desafio. A partir dela, você criará uma nova branch para implementar a regra de negócio e configurar a documentação viva.

No terminal, escolha a pasta onde deseja salvar o projeto e execute:

```
git clone -b develop git@github.com:IA-para-DEVs-SCTEC-T2/bookshelf-api.git
```

Depois entre na pasta do projeto:

```
cd bookshelf-api
```

Confirme se você está na branch `develop`:

```
git branch
```

A saída deve mostrar algo parecido com:

```
* develop
```

Atualize a branch `develop` com a versão mais recente do repositório remoto:

```
git pull origin develop
```

Crie a branch do desafio:

```
git checkout -b feature/bookshelf-live-docs
```

Confirme se você está na nova branch:

```
git branch
```

A branch ativa deve aparecer com um `*`, por exemplo:

```
develop  
* feature/bookshelf-live-docs
```

2. Criar a pasta de prompts

Nesta etapa, você vai criar a pasta onde ficará o prompt template usado pela documentação viva.

Na raiz do projeto, execute:

```
mkdir -p docs/prompts
```

Agora crie o arquivo do prompt:

```
mkdir -p docs/prompts
```

Esse arquivo será o template fixo. Ele terá instruções gerais para a IA analisar o diff e decidir quais documentos precisam ser atualizados.

3. Adicionar o prompt template

Nesta etapa, você vai criar o prompt que será reutilizado sempre que uma mudança no código precisar ser documentada.

Abra o arquivo:

```
docs/prompts/docs-live-update.md
```

Cole o conteúdo abaixo:

```
# Documentação viva — atualização por diff

Analise o Git diff abaixo e atualize apenas a documentação impactada.

## Arquivos permitidos

Você pode alterar somente:

- `README.md`
- `docs/INSTALLATION.md`
- `docs/diagrams/api-flow.md`

Não altere outros arquivos.
Não crie novos documentos.
Não altere `openapi.yaml`.

## Regras gerais

- Use somente mudanças reais presentes no diff.
- Não invente endpoints, comandos, dependências, status codes, mensagens ou fluxos.
- Preserve o estilo atual dos documentos.
- Faça alterações pontuais, sem reescrever arquivos inteiros.
- Se um documento não for impactado, mantenha-o inalterado.
- Ignore contexto anterior e use apenas a seção `Diff detectado`.

## Critérios de atualização

Atualize `README.md` quando houver:
```

- novo endpoint ou alteração em endpoint existente;
- nova funcionalidade visível;
- nova regra de negócio;
- nova validação;
- alteração em status code, mensagem de erro ou exemplo de uso;
- novo comando ou script relevante.

Atualize ``docs/INSTALLATION.md`` somente quando houver:

- nova dependência;
- alteração em instalação;
- alteração em comando de execução;
- alteração em variável de ambiente;
- alteração de porta;
- alteração no processo de setup.

Não altere ``docs/INSTALLATION.md`` apenas por nova rota, validação ou regra de negócio.

Atualize ``docs/diagrams/api-flow.md`` quando houver:

- novo endpoint;
- alteração em rota existente;
- alteração em método HTTP;
- alteração em status code;
- alteração em request, response ou mensagem de erro;
- nova validação ou regra de negócio;
- alteração no fluxo entre cliente, API e resposta.

Diagramas Mermaid

Se a mudança afetar fluxo de API, rota, validação ou regra de negócio, atualize ``docs/diagrams/api-flow.md`` com um diagrama Mermaid do tipo ``sequenceDiagram``.

O diagrama deve ser dinâmico e derivado do diff.

Para montar o diagrama:

- identifique a rota alterada, como ``app.get``, ``app.post``, ``app.patch``, ``app.put`` ou ``app.delete``;
- identifique o método e o caminho da rota;
- identifique condições reais do código, como ``if``, validações e regras de negócio;
- identifique retornos reais, como ``return res.status(...).json(...)`` ou ``return res.status(...).send(...)``;
- crie branches ``alt`` e ``else`` somente para condições que aparecem no código;
- use somente status codes e mensagens presentes no diff;
- represente alteração de estado apenas se ela existir no código.

Não inclua automaticamente ``400``, ``404``, ``409``, ``200``, ``201`` ou ``204``.
Inclua esses status apenas se aparecerem no diff.

Se já existir diagrama para a rota afetada, atualize-o.

Se não existir, crie uma seção para a rota ou fluxo impactado.

Saída esperada

Atualize diretamente os documentos necessários.

Ao final, informe brevemente:

- mudanças detectadas;
- documentos alterados;
- documentos preservados;
- diagramas criados ou atualizados;
- pontos para revisão humana, se houver.

Arquivos modificados

```
```txt
{{CHANGED_FILES}}
```
```

Diff detectado

```
```diff
{{GIT_DIFF}}
```
```

Os campos abaixo são placeholders:

```
{{CHANGED_FILES}}
{{GIT_DIFF}}
```

Eles serão substituídos automaticamente pelo script.

4. Criar o script de documentação viva

Nesta etapa, você vai criar um script que coleta o `git diff`, injeta esse diff no prompt template e gera um arquivo final para ser usado no Kiro.

Crie a pasta de scripts:

```
mkdir -p scripts
```

Crie o arquivo:

```
scripts/docs-live.sh
```

Abra:

```
scripts/docs-live.sh
```

Cole o conteúdo abaixo:

```
#!/usr/bin/env bash

set -e

REPO_ROOT="$(git rev-parse --show-toplevel)"
cd "$REPO_ROOT"

PROMPT_TEMPLATE="docs/prompts/docs-live-update.md"
OUTPUT_DIR=".tmp/docs-live"
OUTPUT_PROMPT="$OUTPUT_DIR/docs-live.generated.md"

mkdir -p "$OUTPUT_DIR"

if [ ! -f "$PROMPT_TEMPLATE" ]; then
    echo "Erro: template não encontrado em $PROMPT_TEMPLATE"
    exit 1
fi

python3 <<'PY'
from pathlib import Path
import subprocess
import sys

repo_root = Path.cwd()

template_path = repo_root / "docs/prompts/docs-live-update.md"
output_path = repo_root / ".tmp/docs-live/docs-live.generated.md"

def run_git_command(args):
    result = subprocess.run(
        ["git"] + args,
        cwd=repo_root,
        text=True,
        capture_output=True
    )

    if result.returncode != 0:
        print(result.stderr)
        sys.exit(result.returncode)

    return result.stdout.strip()

working_diff = run_git_command(["diff"])
```

```

staged_diff = run_git_command(["diff", "--cached"])

working_files = run_git_command(["diff", "--name-only"])
staged_files = run_git_command(["diff", "--cached", "--name-only"])

all_files = []
for item in (working_files + "\n" + staged_files).splitlines():
    item = item.strip()
    if item and item not in all_files:
        all_files.append(item)

git_diff = "\n\n".join(part for part in [working_diff, staged_diff] if part.strip())
changed_files = "\n".join(all_files)

if not git_diff.strip():
    print("Nenhuma mudança detectada pelo Git.")
    print("Altere algum arquivo antes de gerar a documentação viva.")
    sys.exit(0)

content = template_path.read_text(encoding="utf-8")
content = content.replace("{{CHANGED_FILES}}", changed_files)
content = content.replace("{{GIT_DIFF}}", git_diff)

output_path.parent.mkdir(parents=True, exist_ok=True)
output_path.write_text(content, encoding="utf-8")

print()
print("Prompt gerado com sucesso:")
print(output_path)
print()
print("Use este arquivo no Kiro para atualizar a documentação viva.")
PY

```

Dê permissão de execução:

```
chmod +x scripts/docs-live.sh
```

5. Ignorar arquivos temporários

Nesta etapa, você vai evitar que os arquivos temporários gerados pelo script sejam versionados no Git.

Abra ou crie o arquivo:

```
.gitignore
```

Adicione:

```
.tmp/
```

A pasta `.tmp/` será usada apenas para armazenar o prompt gerado automaticamente.

6. Adicionar o comando no `package.json`

Nesta etapa, você vai criar um comando para executar a documentação viva com npm.

Abra o arquivo:

```
package.json
```

Dentro de `"scripts"`, adicione:

```
"docs:live": "bash ../scripts/docs-live.sh"
```

Exemplo:

```
{
  "scripts": {
    "start": "node src/server.js",
    "test": "jest",
    "build": "node -c src/app.js && node -c src/server.js",
    "lint": "eslint src tests",
    "docs:live": "bash ../scripts/docs-live.sh"
  }
}
```

Se o seu projeto tiver outro conjunto de scripts, apenas acrescente o `docs:live` sem remover os existentes.

7. Testar se o script funciona

Nesta etapa, você vai verificar se o comando foi configurado corretamente.

Entre na pasta **backend** do projeto, execute:

```
npm run docs:live
```

Se ainda não houver mudanças no código, a saída esperada será algo parecido com:

Nenhuma mudança detectada pelo Git.

Altere algum arquivo antes de gerar a documentação viva.

Isso significa que o script está funcionando.

8. Alterar a regra de negócio no BookShelf-API

Nesta etapa, você vai implementar a mudança principal do desafio.

A regra nova será:

Livros com status reading não podem ser removidos diretamente.

A rota impactada é:

```
DELETE /books/:id
```

Abra o arquivo:

```
src/app.js
```

Procure a rota atual:

```
app.delete('/books/:id', (req, res) => {  
  const book = findBookById(req.params.id);  
  
  if (!book) {  
    return res.status(404).json({  
      error: 'Livro não encontrado'  
    });  
  }  
}
```

```
books = books.filter((item) => item.id !== book.id);

return res.status(204).send();
});
```

Substitua por:

```
app.delete('/books/:id', (req, res) => {
  const book = findBookById(req.params.id);

  if (!book) {
    return res.status(404).json({
      error: 'Livro não encontrado'
    });
  }

  if (book.status === 'reading') {
    return res.status(409).json({
      error: 'Livro em leitura não pode ser removido diretamente'
    });
  }

  books = books.filter((item) => item.id !== book.id);

  return res.status(204).send();
});
```

O que mudou:

Antes:

qualquer livro existente podia ser removido.

Depois:

livros com status reading não podem ser removidos diretamente.

O novo comportamento esperado é:

```
404 → livro não encontrado
409 → livro está em leitura
204 → livro removido com sucesso
```

9. Gerar o prompt com o diff da mudança

Nesta etapa, você vai gerar o prompt automático com base na alteração feita no código.

Dentro da pasta **backend**, execute:

```
npm run docs:live
```

O script deve gerar:

.tmp/docs-live/docs-live.generated.md

Esse arquivo contém:

```
instruções do prompt  
arquivos modificados  
diff detectado
```

10. Conferir se o prompt gerado está correto

Nesta etapa, você vai verificar se o arquivo gerado contém a mudança real do projeto.

Abra:

```
.tmp/docs-live/docs-live.generated.md
```

Verifique se aparece a seção:

```
## Arquivos modificados
```

Ela deve indicar algo como:

```
src/app.js
```

Depois verifique a seção:

```
## Diff detectado
```

Ela deve mostrar a nova regra adicionada:

```
+ if (book.status === 'reading') {  
+   return res.status(409).json({  
+     error: 'Livro em leitura não pode ser removido diretamente'  
+   });  
+ }
```

Se o diff aparecer, o prompt foi gerado corretamente.

11. Usar o prompt no Kiro ou GitHub Copilot

Nesta etapa, você vai pedir para a IA atualizar apenas a documentação impactada.

No Kiro ou GitHub Copilot, use este prompt:

Abra e leia novamente o arquivo .tmp/docs-live/docs-live.generated.md antes de fazer qualquer alteração.

Use exclusivamente o diff presente na seção "Diff detectado".

Atualize somente a documentação impactada entre:

- README.md
- docs/INSTALLATION.md
- docs/diagrams/api-flow.md

Se a mudança afetar uma rota, validação, regra de negócio ou fluxo da API, atualize docs/diagrams/api-flow.md com um diagrama Mermaid derivado dos retornos reais encontrados no diff.

Não altere outros arquivos.

Não crie novos documentos.

Não altere openapi.yaml nesta etapa.

Ignore contexto de execuções anteriores.

A IA deve analisar o diff e decidir quais documentos foram impactados.

12. O que deve ser atualizado

Nesta etapa, você vai entender o que provavelmente deve ser modificado na documentação.

Como a mudança afeta a rota:

```
DELETE /books/:id
```

A documentação provavelmente deve atualizar:

```
README.md  
docs/diagrams/api-flow.md
```

O docs/INSTALLATION.md provavelmente não deve ser alterado, porque a mudança não envolve:

```
nova dependência  
novo comando de instalação  
nova variável de ambiente  
nova porta  
novo processo de setup
```

README.md

O README pode receber uma descrição como:

```
- `DELETE /books/:id` — remove um livro por ID. Retorna `409 Conflict` quando o livro está com status `reading`, `404` quando o livro não é encontrado e `204 No Content` quando a remoção é concluída com sucesso.
```

docs/diagrams/api-flow.md

O diagrama deve representar o fluxo real da rota `DELETE /books/:id`.

Exemplo esperado:

```
## Remoção de livro  
  
Este fluxo representa a rota `DELETE /books/:id`.  
  
```mermaid  
sequenceDiagram
 participant Cliente
 participant API
```

```
participant Memoria as Memória (books)
```

```
Cliente-->>API: DELETE /books/:id
```

```
API-->>Memoria: Busca livro pelo ID
```

```
alt Livro não encontrado
```

```
 API-->>Cliente: 404 Livro não encontrado
```

```
else Livro em leitura
```

```
 API-->>Cliente: 409 Livro em leitura não pode ser removido diretamente
```

```
else Livro removível
```

```
 API-->>Memoria: Remove livro da lista
```

```
 API-->>Cliente: 204 No Content
```

```
end
```

```
...
```

Esse diagrama deve ser criado ou ajustado conforme a estrutura atual do arquivo.

---

## 13. Revisar o que a IA alterou

Nesta etapa, você vai revisar se a documentação gerada está coerente com o código.

Execute:

```
git diff
```

Verifique:

```
se o README menciona a regra do status reading
se o api-flow representa 404, 409 e 204 corretamente
se o INSTALLATION foi preservado quando não havia impacto de setup
se a IA não alterou arquivos fora do escopo
se o diagrama Mermaid não inventou status codes ou fluxos inexistentes
```

Se algum arquivo foi alterado indevidamente, restaure esse arquivo.

Exemplo:

```
git restore openapi.yaml
```

---

## 14. Testar a aplicação

Nesta etapa, você vai validar se a regra de negócio realmente funciona.

Instale dependências, se necessário:

```
npm install
```

Rode os testes, se existirem:

```
npm test
```

Rode lint, se existir:

```
npm run lint
```

Rode build ou checagem de sintaxe, se existir:

```
npm run build
```

Inicie a API:

```
npm start
```

---

## 15. Testar a regra com curl

Nesta etapa, você vai testar os três comportamentos esperados da rota DELETE /books/:id.

Pelo código base, o livro de ID 1 está com status reading.

Teste:

```
curl -i -X DELETE http://localhost:3000/books/1
```

Se o seu projeto usa outra porta, substitua 3000 pela porta configurada no server.js.

Resposta esperada:

```
HTTP/1.1 409 Conflict
```

Com corpo parecido com:

```
{
 "error": "Livro em leitura não pode ser removido diretamente"
}
```

Agora teste um livro que pode ser removido, como o ID 2, que está com status unread:

```
curl -i -X DELETE http://localhost:3000/books/2
```

Resposta esperada:

```
HTTP/1.1 204 No Content
```

Como a resposta é 204, ela não deve retornar corpo.

Agora teste um ID inexistente:

```
curl -i -X DELETE http://localhost:3000/books/999
```

Resposta esperada:

```
HTTP/1.1 404 Not Found
```

Com corpo parecido com:

```
{
 "error": "Livro não encontrado"
}
```

---

## 16. Conferir se a documentação ficou coerente



Nesta etapa, você vai confirmar se a documentação acompanha o comportamento real da API.

No README.md, verifique se aparece:

```
DELETE /books/:id
409 quando o livro está em leitura
404 quando o livro não existe
204 quando o livro é removido
```

No docs/diagrams/api-flow.md, verifique se aparece o fluxo de remoção de livro com:

```
404 Livro não encontrado
409 Livro em leitura
204 No Content
```

No docs/INSTALLATION.md, verifique se ele só foi alterado caso exista impacto real em instalação ou execução.

---

## 17. Fazer commit da mudança

Nesta etapa, você vai versionar código e documentação juntos.

Verifique o status:

```
git status
```

Adicione os arquivos:

```
git add .
```

Faça o commit:

```
git commit -m "feat: bloquear a remocao de livros com status reading"
```

Envie para o GitHub:

```
git push origin feature/bookshelf-live-docs
```

---

## 18. Fluxo final da prática

Nesta etapa, você consolida o fluxo aplicado no desafio.

O processo completo foi:

1. Criar o prompt template
2. Criar o script docs-live
3. Configurar o comando `npm run docs:live`
4. Alterar a regra de negócio no código
5. Gerar o prompt com o git diff
6. Usar a IA para atualizar a documentação impactada
7. Revisar o diff gerado pela IA
8. Testar a API
9. Commitar código e documentação juntos

---

## 19. Resumo do que foi aprendido

Nesta etapa, você revisa o principal aprendizado da prática.

Com esse fluxo, você aprendeu a:

gerar documentação viva a partir de mudanças reais no código  
usar git diff como fonte da verdade para a IA  
criar um prompt template reutilizável  
atualizar README e diagramas com base no impacto da mudança  
preservar documentos que não foram impactados  
revisar tecnicamente a documentação gerada por IA  
versionar código e documentação no mesmo ciclo

A ideia central é:

Documentação viva não é atualizar tudo.  
É atualizar apenas o que foi impactado pela mudança.